Networks and Internet Programming

Transmission Control Protocol

Eng. Asma Abdel Karim
Computer Engineering Department

Outline

- Overview.
- Advantages of TCP Over UDP.
- Communication between Applications Using Ports.
- Socket Operations.
- TCP and the Client/Server Paradigm.
- TCP Sockets and Java.
- Socket Class.
- Creating a TCP Client.
- ServerSocket Class.
- Creating a TCP Server.
- Exception Handling: Socket-Specific Exceptions.

Eng. Asma Abdel Karim
Computer Engineering Department

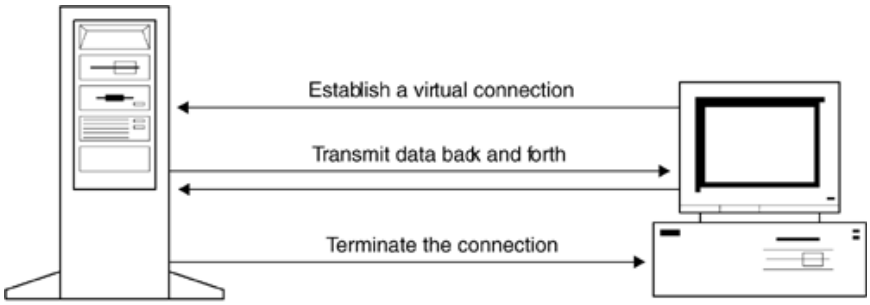
Overview

- The properties of TCP make it highly attractive to network programmers.
  - As it simplifies network communication by removing many of the obstacles of UDP, such as ordering of packets and packet loss.
- UDP is concerned with the transmission of packets of data.
  - TCP focuses instead on establishing a network connection, through which a stream of bytes may be sent and received.

Overview (Cont.)

- Packets may be sent through a network using various paths and may arrive at different times.
- This benefits performance and robustness, as the loss of a single packet doesn't necessarily disrupt the transmission of other packets.
- Nonetheless, such a system creates extra work for programmers who need to guarantee delivery of data.
- TCP eliminates this extra work by guaranteeing delivery and order, providing for a reliable byte communication stream between client and server that supports two-way communication.
- TCP establishes a "virtual connection" between two machines, through which streams of data may be sent.

Overview (Cont.)



Overview (Cont.)

- TCP uses a lower-level communications protocol, the ***Internet Protocol (IP)***, to establish the connection between machines.
  - This connection provides an interface that allows streams of bytes to be sent and received, and transparently converts the data into IP datagram packets.
- TCP provides guaranteed delivery of bytes of data.
  - Of course, it's always possible that network errors will prevent delivery, but TCP handles the implementation issues such as resending packets, and alerts the programmer only in serious cases such as if there is no route to a network host or if a connection is lost.

Overview (Cont.)

- The virtual connection between two machines is represented by a ***socket***.
- There are substantial differences between a UDP socket and a TCP socket.
  - First, TCP sockets are connected to a single machine, whereas UDP sockets may transmit or receive data from multiple machines.
  - Second, UDP sockets only send and receive ***packets*** of data, whereas TCP allows transmission of data through ***byte streams*** (represented as an InputStream and OutputStream). They are converted into datagram packets for transmission over the network, without requiring the programmer to intervene.

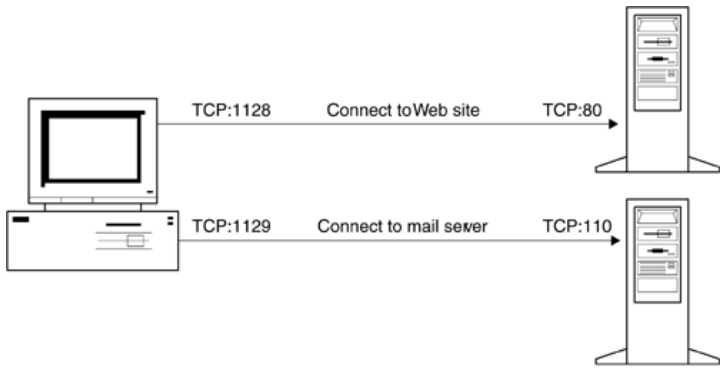
Advantages of TCP over UDP

- Automatic Error Control.
- Reliability.
- Ease of Use.

Communication between Applications Using Ports

- It is clear that there are significant differences between TCP and UDP, but there is also an important similarity between these two protocols. Both share the concept of a communications port, which distinguishes one application from another.
- When a TCP socket establishes a connection to another machine, it requires two very important pieces of information to connect to the remote end—the IP address of the machine and the port number.
- In addition, a local IP address and port number will be bound to it, so that the remote machine can identify which application established the connection.

Communication between Applications Using Ports (Cont.)



Communication between Applications Using Ports (Cont.)

- Ports in TCP are just like ports in UDP—they are represented by a number in the range 1–65535.
- Ports below 1024 are restricted to use by well-known services such as HTTP, FTP, SMTP, POP3, and telnet.

Well-Known Services	Service Port
Telnet	23
Simple Mail Transfer Protocol	25
HyperTexr Transfer Protocol	80
Post Office Protocol 3	110

Socket Operations

- TCP sockets can perform a variety of operations. They can:
  - Establish a connection to a remote host.
  - Send data to a remote host.
  - Receive data from a remote host.
  - Close a connection.
- In addition, there is a special type of socket that provides a service that will bind to a specific port number. This type of socket is normally used only in servers, and can perform the following operations:
  - Bind to a local port.
  - Accept incoming connections from remote hosts.
  - Unbind from a local port.
- These two sockets are grouped into different categories, and are used by either a client or a server (since some clients may also be acting as servers, and some servers as clients). However, it is normal practice for the role of client and server to be separate.

TCP and the Client/Server Paradigm

- The client/server paradigm divides software into two categories, clients and servers.
  - A client is software that initiates a connection and sends requests, whereas
  - A server is software that listens for connections and processes requests.
- In the context of UDP programming, no actual connection is established, and UDP applications may both initiate and receive requests on the same socket.
- In the context of TCP, where connections are established between machines, the client/server paradigm is much more relevant.

TCP and the Client/Server Paradigm (Cont.)

- When software acts as a client, or as a server, it has a rigidly defined role that fits easily into a familiar mental model.
  - Either the software is initiating requests, or it is processing them.
- Switching between these roles makes for a more complex system.
  - Even if switching is permitted, at any given time one software program must be the client and one software program must be the server. If they both try to be clients at the same time, no server exists to process the requests!

Network Clients

- Network clients initiate connections and usually take charge of network transactions.
- The server is there to fulfill the requests of the client—a client does not fulfill the requests of a server.
- Although the client is in control, some power still resides in the server, of course. A client can tell a server to delete all files on the local file system, but the server isn't necessarily compelled to carry out that action.
- The network client speaks to the server using an agreed-upon standard for communication, the network protocol.
  - For example, an HTTP client uses a set of commands different from a mail client, and has a completely different purpose.
  - Connecting an HTTP client to a mail server, or a mail client to an HTTP server, will result not only in an error message but in an error message that the client will not understand.
  - For this reason, as part of the protocol specification, a port number is used so that the client can locate the server.

Network Servers

- The role of the network server is to bind to a specific port (which is used by the client to locate the server), and to listen for new connections.
- While the client is temporary, and runs only when the user chooses, the server must run continually (even if no clients are actually connected) in the hope that someone, at some time, will want its services.
- Some servers can handle only a single connection at a time, while others can handle many connections concurrently, through the use of threads.

TCP Sockets and Java

- Java offers good support for TCP sockets, in the form of two socket classes, ***java.net.Socket*** and ***java.net.ServerSocket***.
- When writing client software that connects to an existing service, the ***Socket*** class should be used.
- When writing server software that binds to a local port in order to provide a service, the ***ServerSocket*** class should be employed.
- This is different from the way a ***DatagramSocket*** works with UDP.
  - The function of connecting to servers, and the function of accepting data from clients, is split into a separate class under TCP.

Socket Class

- The ***Socket*** class represents client sockets, and is a communication channel between two TCP communications ports belonging to one or two machines.
- A socket may connect to a port on the local system, avoiding the need for a second machine, but most network software will usually involve two machines.
- TCP sockets can't communicate with more than two machines, however.
  - If this functionality is required, a client application should establish multiple socket connections, one for each machine.

Socket Class - Constructors

- The easiest way to create a socket is to specify the hostname of the machine and the port of the service. For example, to connect to a Web server on port 80, the following code might be used:

```
try{
        // Connect to the specified host and port
        Socket mySocket = new Socket ( "www.awl.com", 80);
        // ......
}
catch (Exception e){
        System.err.println ("Err – " + e);
}
```

Socket Class – Constructors (Cont.)

- **protected Socket ()**
  - *Creates an unconnected socket using the default implementation provided by the current socket factory. Developers should not normally use this constructor , as it does not allow a hostname or port to be specified.*
- **Socket (InetAddress address, int port) throws java.io.IOException, java.lang.SecurityException**
  - *Creates a socket connected to the specified IP address and port. If a connection cannot be established, or if connecting to that host violates a security restriction (such as when an applet tries to connect to a machine other than the machine from which it was loaded), an exception is thrown.*
- **Socket (InetAddress address, int port, InetAddress localAddress, int localPort) throws java.io.IOException, java.lang.SecurityException**
  - *Creates a socket connected to the specified address and port, and is bound to the specified local address and local port. By default, a free port is used, but this method allows you to specify a specific port number, as well as a specific address, in the case of multihomed hosts (i.e., a machine where the localhost is known by two or more IP addresses).*

Socket Class – Constructors (Cont.)

- **protected Socket (SocketImpl implementation)**
  - Creates an unconnected socket using the specified socket implementation. Developers should not normally use this constructor, as it does not allow a hostname or port to be specified.
- **Socket (String host, int port) throws java.net.UnknownHostException, java.io.IOException, java.lang.SecurityException**
  - Creates a socket connected to the specified host and port. This method allows a string to be specified, rather than an InetAddress. If the hostname could not be resolved, a connection could not be established, or a security restriction is violated, an exception is thrown.
- **Socket (String host, int port, InetAddress localAddress, int localPort) throws java.net.UnknownHostException, java.io. IOException, java.lang.SecurityException**
  - Creates a socket connected to the specified host and port, and bound to the specified local port and address. This allows a hostname to be specified as a string, and not an InetAddress instance, as well as allowing a specific local address and port to be bound to. These local parameters are useful for multihomed hosts (i.e., a machine where the localhost is known by two or more IP addresses). If the hostname can't be resolved, a connection cannot be established, or a security restriction is violated, an exception is thrown.

Creating a Socket

- Under normal circumstances, a socket is connected to a machine and port when it is created.
  - Although there is a blank constructor that does not require a hostname or port, it is protected and can't be called from normal applications.
  - Furthermore, there isn't a connect() method that allows you to specify these details at a later point in time, so under normal circumstances the socket will be connected when created.
- If the network is fine, the call to a socket constructor will return as soon as a connection is established, but if the remote machine is not responding, the constructor method may block for an indefinite amount of time.
  - This varies from system to system, depending on a variety of factors such as the operating system being used and the default network timeout.
  - In mission-critical systems it may be appropriate to place such calls in a second thread, to prevent an application from stalling.

Using a Socket

- **void close() throws java.io.IOException**
  - *Closes the socket connection. Closing a connect may or may not allow remaining data to be sent, depending on the streams before closing a socket connection.*
- **InetAddress getInetAddress()**
  - *Returns the address of the remote machine that is connected to the socket.*
- **InputStream getInputStream() throws java.io.IOException**
  - *Returns an input stream, which reads from the application this socket is connected to.*
- **OutputStream getOutputStream() throws java.io.IOException**
  - *Returns an output stream, which writes to the application that this socket is connected to.*

Using a Socket (Cont.)

- **boolean getKeepAlive() throws java.net.SocketException**
  - *Returns the state of the SO_KEEPALIVE socket option.*
- **InetAddress getLocalAddress()**
  - *Returns the local address associated with the socket (useful in the case of multihomed machines).*
- **int getLocalPort()**
  - *Returns the port number that the socket is bound to on the local machine.*
- **int getPort()**
  - *Returns the port number of the remote service to which the socket is connected.*
- **int getReceiveBufferSize() throws java.net.SocketException**
  - *Returns the receive buffer size used by the socket, determined by the value of the SO_RCVBUF socket option.*
- **int getSendBufferSize() throws java.net.SocketException**
  - *Returns the send buffer size used by the socket, determined by the value of the SO_SNDBUF socket option.*

Using a Socket (Cont.)

- **int getSoLinger() throws java.net.SocketException**
  - *Returns the value of the SO_LINGER socket option, which controls how long unsent data will be queued when a connection is terminated.*
- **int getSoTimeout() throws java.net.SocketException**
  - *Returns the value of the SO_TIMEOUT socket option, which controls how many milliseconds a read operation will block for. If a value of 0 is returned, the timer is disabled and a thread will block indefinitely (until data is available or the stream is terminated).*
- **boolean getTcpNoDelay() throws java.net.SocketException**
  - *Returns "true" if the TCP_NODELAY socket option is set, which controls whether Nagle's algorithm is enabled.*

Using a Socket (Cont.)

- **void setKeepAlive(boolean onFlag) throws java.net.SocketException**
  - *Enables or disables the SO_KEEPALIVE socket option.*
- **void setReceiveBufferSize(int size) throws java.net.SocketException**
  - *Modifies the value of the SO_RCVBUF socket option, which recommends a buffer size for the operating system's network code to use for receiving incoming data. Not every system will support this functionality or allows absolute control over this feature. If you want to buffer incoming data, you're advised to instead use a BufferedInputStream or a BufferedReader.*
- **void setSendBufferSize(int size) throws java.net.SocketException**
  - *Modifies the value of the SO_SNDBUF socket option, which recommends a buffer size for the operating system's network code to use for sending incoming data. Not every system will support this functionality or allows absolute control over this feature. If you want to buffer incoming data, you're advised to instead use a BufferedOutputStream or a BufferedWriter.*

Using a Socket (Cont.)

- **static void setSocketImplFactory (SocketImplFactory factory) throws java.net.SocketException, java.io.IOException java.lang.SecurityException**
  - *Assigns a socket implementation factory for the JVM, which may already exist, or may violate security restrictions, either of which causes an exception to be thrown. Only one factory can be specified, and this factory will be used whenever a socket is created.*
- **void setSoLinger(boolean onFlag, int duration) throws java.net.SocketException java.lang.IllegalArgumentException**
  - *Enables or disables the SO_LINGER socket option (according to the value of the onFlag boolean parameter), and specifies a duration in seconds. If a negative value is specified, an exception is thrown.*

Using a Socket (Cont.)

- **void setSoTimeout(int duration) throws java.net.SocketException**
  - *Modifies the value of the SO_TIMEOUT socket option, which controls how long (in milliseconds) a read operation will block. A value of zero disables timeouts, and blocks indefinitely. If a timeout does occur, a java.io.IOInterruptedException is thrown whenever a read operation occurs on the socket's input stream. This is distinct from the internal TCP timer, which triggers a resend of unacknowledged datagram packets.*
- **void setTcpNoDelay(boolean onFlag) throws java.net.SocketException**
  - *Enables or disables the TCP_NODELAY socket option, which determines whether Nagle's algorithm is used.*

Using a Socket (Cont.)

- **void shutdownInput() throws java.io.IOException**
  - *Closes the input stream associated with this socket and discards any further information that is sent. Further reads to the input stream will encounter the end of the stream marker.*
- **void shutdownOutput() throws java.io.IOException**
  - *Closes the output stream associated with this socket. Any data previously written, but not yet sent, will be flushed, followed by a TCP connection-termination sequence, which notifies the application that no more data will be available (and in the case of a Java application, that the end of the stream has been reached). Further writes to the socket will cause an IOException to be thrown.*

Reading from and Writing to TCP Sockets

- Once a socket is created, it is connected and ready to read/write by using the socket's input and output streams.
- These streams don't need to be created; they are provided by the ***Socket.getInputStream()*** and ***Socket.getOutputStream()*** methods.
- A filter can easily be connected to a socket stream, to make for simpler programming.

Reading from and Writing to TCP Sockets (Example)

```
try{
        // Connect a socket to some host machine and port
        Socket socket = new Socket ( somehost, someport );
        // Connect a buffered reader
        BufferedReader reader = new BufferedReader ( new
        InputStreamReader ( socket.getInputStream() ) );
        // Connect a print stream
        PrintStream pstream = new PrintStream(
                        socket.getOutputStream() );
}
catch (Exception e){
        System.err.println ("Error – " + e);
}
```

Socket Options

- Socket options are settings that modify how sockets work, and they can affect (both positively and negatively) the performance of applications.
- Generally, socket options should not be changed unless there is a good reason for doing so, as changes may negatively affect application and network performance.
- The one exception to this caveat is the SO_TIMEOUT option.
  - Virtually every TCP application should handle timeouts gracefully rather than stalling if the application the socket is connected to fails to transmit data when required.

SO_KEEPALIVE Socket Option

- By default, no data is sent between two connected sockets unless an application has data to send.
  - This means that an idle socket may not have data submitted for minutes, hours, or even days in the case of long-lived processes.
- Suppose, however, that a client crashes, and the end-of-connection sequence is not sent to a TCP server.
  - Valuable resources (CPU time and memory) might be wasted on a client that will never respond.
- When the keepalive socket option is enabled, the other end of the socket is probed to verify it is still active.
  - However, the application doesn't have any control over how often keepalive probes are sent.

SO_KEEPALIVE Socket Option (Cont.)

- To enable ***keepalive***, the ***Socket.setSoKeepAlive(boolean)*** method is called with a value of "true" (a value of "false" will disable it).
- For example, to enable ***keepalive*** on a socket, the following code would be used.

```
// Enable SO_KEEPALIVE
someSocket.setSoKeepAlive(true);
```

- It should also be kept in mind that keepalive doesn't allow you to specify a value for probing socket endpoints.
  - A better solution than keepalive, and one that developers are advised to use, is to instead modify the timeout socket option.

SO_RCVBUF Socket Option

- The receive buffer socket option controls the buffer used for receiving data.
- Changes can be made to the size by calling the ***Socket.setReceiveBufferSize(int)*** method.
- For example, to increase the receive buffer size to 4,096 bytes, the following code would be used.

```
// Modify receive buffer size
someSocket.setReceiveBufferSize(4096);
```

SO_RCVBUF Socket Option (Cont.)

- Note that a request to modify the size of the receive buffer does not guarantee that it will change.
- For example, some operating systems may not allow this socket option to be modified, and will ignore any changes to the value.
- The current buffer size can be determined by invoking the ***Socket.getReceiveBufferSize()*** method.
- A better choice for buffering is to use a ***BufferedInputStream***/***BufferedReader***.

SO_SNDBUF Socket Option

- The send buffer socket option controls the size of the buffer used for sending data.
- By calling the ***Socket.setSendBufferSize(int)*** method, you can attempt to change the buffer size, but requests to change the size may be rejected by the operating system.

```
//Set the send buffer size to 4096 bytes
someSocket.setSendBufferSize(4096);
```

- To determine the size of the current send buffer, you can call the ***Socket.getSendBufferSize()*** method, which returns an int value.

```
// Get the default size
int size = someSocket.getSendBufferSize();
```

SO_LINGER Socket Option

- When a TCP socket connection is closed, it is possible that data may be queued for delivery and not yet sent (particularly if an IP datagram becomes lost in transit and must be resent).
- The linger socket option controls the amount of time during which unsent data may be sent, after which it is discarded completely.
- It is possible to enable/disable the linger option entirely, or to modify the duration of a linger, by using the ***Socket.setSoLinger(boolean onFlag, int duration)*** method:

```
// Enable linger, for fifty seconds
someSocket.setSoLinger( true, 50 );
```

TCP_NODELAY Socket Option

- This socket option is a flag, the state of which controls whether Nagle's algorithm (RFC 896) is enabled or not.
- Because TCP data is sent over the network using IP datagrams, a fair bit of overhead exists for each packet, such as IP and TCP header information.
- If only a few bytes at a time are sent in each packet, the size of the header information will far exceed that of the data.
- On a local area network, the extra amount of data sent probably won't amount to much, but on the Internet, where hundreds, thousands, or even millions of clients may be sending such packets through individual routers, this adds up to a significant amount of bandwidth consumption.

TCP_NODELAY Socket Option (Cont.)

- The solution is Nagle's algorithm, which states that TCP may send only one datagram at a time.
- When an acknowledgment comes back for each IP datagram, a new packet is sent containing any data that has been queued up.
- This limits the amount of bandwidth being consumed by packet header information, but at a not insignificant cost—network latency.
- Since data is being queued, it isn't dispatched immediately, so systems that require quick response times such as X-Windows or telnet are slowed.
- Disabling Nagle's algorithm may improve performance, but if used by too many clients, network performance is reduced.

TCP_NODELAY Socket Option (Cont.)

- Nagle's algorithm is enabled or disabled by invoking the ***Socket.setTcpNoDelay (boolean state)*** method.
- For example, to deactivate the algorithm, the following code would be used:

```
// Disable Nagle's algorithm for faster response times
someSocket.setTcpNoDelay(false);
```

- To determine the state of Nagle's algorithm and the TCP_NODELAY flag, the ***Socket.getTcpNoDelay()*** method is used:

```
// Get the state of the TCP_NODELAY flag
boolean state = someSocket.getTcpNoDelay();
```

SO_TIMEOUT Socket Option

- This timeout option is the most useful socket option.
- By default, I/O operations (be the file- or network-based) are blocking.
- An attempt to read data from an InputStream will wait indefinitely until input arrives.
- If the input never arrives, the application stalls and in most cases becomes unusable (unless multithreading is used).
- A more robust application will anticipate such problems and take corrective action.

SO_TIMEOUT Socket Option (Cont.)

- When the SO_TIMEOUT option is enabled, any read request to the InputStream of a socket starts a timer.
- When no data arrives in time and the timer expires, a ***java.io.InterruptedIOException*** is thrown, which can be caught to check for a timeout.
- What happens then is up to the application developer— a retry attempt might be made, the user might be notified, or the connection aborted.
- The duration of the timer is controlled by calling the ***Socket.setSoTimeout(int)*** method, which accepts as a parameter the number of milliseconds to wait for data.

SO_TIMEOUT Socket Option (Cont.)

- For example, to set a five-second timeout, the following code would be used:

```
// Set a five second timeout
someSocket.setSoTimeout ( 5 * 1000 );
```

- Once enabled, any attempt to read could potentially throw an ***InterruptedIOException***, which is extended from the ***java.io.IOException*** class.
- Since read attempts can already throw an IOException, no further code is required to handle the exception
  - However, some applications may want to specifically trap timeout-related exceptions, in which case an additional exception handler may be added.

SO_TIMEOUT Socket Option (Cont.)

```
try{
        Socket s = new Socket (...);
        s.setSoTimeout ( 2000 );
        // do some read operation ....
}
catch (InterruptedIOException iioe){
        timeoutFlag = true; // do something special like set a flag
}
catch (IOException ioe){
        System.err.println ("IO error " + ioe);
        System.exit(0);
}
```

SO_TIMEOUT Socket Option (Cont.)

- To determine the length of the TCP timer, the ***Socket.getSoTimeout()*** method, which returns an int, can be used.
- A value of zero indicates that timeouts are disabled, and read operations will block indefinitely.

```
// Check to see if timeout is not zero
if ( someSocket.getSoTimeout() == 0)
        someSocket.setSoTimeout (500);
```

Creating a TCP Client

```
import java.net.*
import java.io.*;
public class DaytimeClient{
    public static final int SERVICE_PORT = 13;
    public static void main(String args[]){
        // Check for hostname parameter
        if (args.length != 1){
            System.out.println ("Syntax - DaytimeClient host");
            return;
        }
        // Get the hostname of server
        String hostname = args[0];
        try{
            // Get a socket to the daytime service
            Socket daytime = new Socket (hostname, SERVICE_PORT);
```

Creating a TCP Client

```
            System.out.println ("Connection established");
            // Set the socket option just in case server stalls
            daytime.setSoTimeout ( 2000 );
            // Read from the server
            BufferedReader reader = new BufferedReader (
                new InputStreamReader(daytime.getInputStream()));
            System.out.println ("Results : " +reader.readLine());
            // Close the connection
            daytime.close();
        }
        catch (IOException ioe){
            System.err.println ("Error " + ioe);
        }
    }
}
```

ServerSocket Class

- A special type of socket, the ***server socket***, is used to provide TCP services.
- Client sockets bind to any free port on the local machine, and connect to a specific server port and host.
- The difference with server sockets is that they bind to a specific port on the local machine, so that remote clients may locate a service.
- Client socket connections will connect to only one machine, whereas server sockets are capable of fulfilling the requests of multiple clients.

ServerSocket Class (Cont.)

- Clients are aware of a service running on a particular port.
- Clients establish a connection, and within the server, the connection is accepted.
  - Multiple connections can be accepted at the same time, or a server may choose to accept only one connection at any given moment.
- Once accepted, the connection is represented as a normal socket, in the form of a ***Socket*** object.
- This ***ServerSocket*** object acts as a factory for client connections, you don't need to create instances of the ***Socket*** class yourself.
  - These connections are modeled as a normal socket, so you can connect input and output filter streams (or even a reader and writer) to the connection.

Creating a ServerSocket

- Once a server socket is created, it will be bound to a local port and ready to accept incoming connections.
- When clients attempt to connect, they are placed into a queue. Once all free space in the queue is exhausted, further clients will be refused.
- The simplest way to create a server socket is to bind to a local address, which is specified as the only parameter, using a constructor.

```
try{
        // Bind to port 80, to provide a TCP service (like HTTP)
        ServerSocket myServer = new ServerSocket ( 80 );
        // ......
}
catch (IOException ioe){
        System.err.println ("I/O error – " + ioe);
}
```

Creating a ServerSocket - Constructors

- **ServerSocket(int port) throws java.io.IOException, java.lang.SecurityException**
  - Binds the server socket to the specified port number, so that remote clients may locate the TCP service.
  - If a value of zero is passed, any free port will be used. However, clients will be unable to access the service unless notified somehow of the port number.
  - By default, the queue size is set to 50, but an alternate constructor is provided that allows modification of this setting.
  - If the port is already bound, or security restrictions (such as security polices or operating system restrictions on well-known ports) prevent access, an exception is thrown.
- **ServerSocket(int port, int numberOfClients) throws java.io.IOException, java.lang.SecurityException**
  - Binds the server socket to the specified port number and allocates sufficient space to the queue to support the specified number of client sockets.
  - If the port is already bound or security restrictions prevent access, an exception is thrown.

Creating a ServerSocket – Constructors (Cont.)

- **ServerSocket(int port, int numberOfClients, InetAddress address) throws java.io.IOException, java.lang.SecurityException**
  - Binds the server socket to the specified port number, and allocates sufficient space to the queue to support the specified number of client sockets.
  - This is an overloaded version of the ServerSocket(int port, int numberOfClients) constructor that allows a server socket to bind to a specific IP address, in the case of a multihomed machine.
  - For example, a machine may have two network cards, or may be configured to represent itself as several machines by using virtual IP addresses.
  - Specifying a null value for the address will cause the server socket to accept requests on all local addresses.
  - If the port is already bound or security restrictions prevent access, an exception is thrown.

Using a ServerSocket

- While the Socket class is fairly versatile, and has many methods, the Server Socket class doesn't really do that much, other than *accept connections* and a*ct as a factory for Socket objects* that model the connection between client and server.
- The most important method is the accept() method, which accepts client connection requests, but there are several others that developers may find useful.

Using a ServerSocket – Methods

- ***Socket accept() throws java.io.IOException, java.lang.SecurityException***
  - Waits for a client to request a connection to the server socket, and accepts it.
  - This is a blocking I/O operation, and will not return until a connection is made (unless the timeout socket option is set).
  - When a connection is established, it will be returned as a Socket object. When accepting connections, each client request will be verified by the default security manager, which makes it possible to accept certain IP addresses and block others, causing an exception to be thrown.
  - However, servers do not need to rely on the security manager to block or terminate connections—the identity of a client can be determined by calling the getInetAddress() method of the client socket.
- ***void close() throws java.io.IOException***
  - Closes the server socket, which unbinds the TCP port and allows other services to use it.

Using a ServerSocket – Methods (Cont.)

- ***InetAddress getInetAddress()***
  - Returns the address of the server socket, which may be different from the local address in the case of a multihomed machine (i.e., a machine whose localhost is known by two or more IP addresses).
- ***int getLocalPort()***
  - Returns the port number to which the server socket is bound.
- ***int getSoTimeout() throws java.io.IOException***
  - Returns the value of the timeout socket option, which determines how many milliseconds an accept() operation can block for. If a value of zero is returned, the accept operation blocks indefinitely.

Using a ServerSocket – Methods (Cont.)

- ***void implAccept(Socket socket) throws java.io.IOException***
  - This method allows ServerSocket subclasses to pass an unconnected socket subclass, and to have that socket object accept an incoming request.
  - Using the implAccept method to accept the connection, an overridden ServerSocket.accept() method can return a connected socket. Few developers will want to subclass the ServerSocket, and using this should be avoided unless required.
- ***static void setSocketFactory (SocketImplFactory factory) throws java.io.IOException, java.net.SocketException, java.lang.Security Exception***
  - Assigns a server socket factory for the JVM. This is a static method, and should be called only once during the lifetime of a JVM. If assigning a new socket factory is prohibited, or one has already been assigned, an exception is thrown.

Using a ServerSocket – Methods (Cont.)

- ***void setSoTimeout(int timeout) throws java.net.SocketException***
  - Assigns a timeout value (specified in milliseconds) for the blocking accept() operation.
  - If a value of zero is specified, timeouts are disabled and the operation will block indefinitely.
  - Providing timeouts are enabled, however, whenever the accept() method is called a timer starts. When the timer expires, a ***java.io.InterruptedIOException*** is thrown, which allows a server to then take further actions.

Accepting and Processing Requests from TCP Clients

- The most important function of a server socket is to accept client sockets.
- Once a client socket is obtained, the server can perform all the "real work" of server programming, which involves reading from and writing to the socket to implement a network protocol.

```
// Perform a blocking read operation, to read the next socket connection
Socket nextSocket = someServerSocket.accept();
// Connect a filter reader and writer to the stream
BufferedReader reader = new BufferedReader (new
        InputStreamReader (nextSocket.getInputStream() ) );
PrintWriter writer = new PrintWriter( new OutputStreamWriter
        (nextSocket.getOutputStream() ) );
```

Creating a TCP Server

```
import java.net.*;
import java.io.*;
public class DaytimeServer{
    public static final int SERVICE_PORT = 13;
    public static void main(String args[]){
        try{
            // Bind to the service port, to grant clients access to the TCP daytime
            //service
            ServerSocket server = new ServerSocket (SERVICE_PORT);
            System.out.println ("Daytime service started");
            // Loop indefinitely, accepting clients
            for (;;) {
                // Get the next TCP client
                Socket nextClient = server.accept();
```

Creating a TCP Server

```
            // Display connection details
            System.out.println ("Received request from " +
                    nextClient.getInetAddress() + ":" +nextClient.getPort() );
            // Don't read, just write the message
            OutputStream out = nextClient.getOutputStream();
            PrintStream pout = new PrintStream (out);
            // Write the current date out to the user
            pout.print( new java.util.Date() );
            // Flush unsent bytes
            out.flush();
            // Close stream
            out.close();
            // Close the connection
            nextClient.close();
        }
    }
```

Creating a TCP Server

```
        catch (BindException be){
            System.err.println ("Service already running on port " + SERVICE_PORT );
        }
        catch (IOException ioe){
            System.err.println ("I/O error - " + ioe);
        }
    }
}
```

Exception Handling: Socket-Specific Exceptions

- All socket-specific exceptions extend from ***SocketException***, so by simply catching that exception, you catch all of the socket-specific ones and write a single generic handler.
- In addition, ***SocketException*** extends from ***java.io.IOException*** if you want to provide a catchall for any I/O exception.
- **SocketException**
  - The ***java.net.SocketException*** represents a generic socket error, which can represent a range of specific error conditions. For finer-grained control, applications should catch its subclasses.

Exception Handling: Socket-Specific Exceptions (Cont.)

- **BindException**
  - The ***java.net.BindException*** represents an inability to bind a socket to a local port. The most common reason for this will be that the local port is already in use.
- **ConnectException**
  - The ***java.net.ConnectException*** occurs when a socket can't connect to a specific remote host and port. There can be several reasons for this, such as that the remote server does not have a service bound to that port, or that it is so swamped by queued connections, it cannot accept any further ones.

Exception Handling: Socket-Specific Exceptions (Cont.)

- **NoRouteToHostException**
  - The ***java.net.NoRouteToHostException*** is thrown when, due to a network error, it is impossible to find a route to the remote host.
  - The cause of this may be local (i.e., the network on which the software application is running), may be a temporary gateway or router problem, or may be the fault of the remote network to which the socket is trying to connect. Another common cause of this is that firewalls and routers are blocking the client software, which is usually a permenant condition.
- **InterruptedIOException**
  - The ***java.net.InterruptedIOException*** occurs when a read operation is blocked for sufficient time to cause a network timeout, as discussed earlier in the chapter. Handling timeouts is a good way to make your code more robust and reliable.

References

Chapter 6 of *Java™ Network Programming and Distributed Computing*, David Reilly and Michael Reilly.